From Detection to Resolution: Using Large Language Models for Autonomous Security Remediation in CI/CD Pipelines

Dr. Ananya Sharma
Department of Computer Science
Stanford University, Stanford, CA, USA

Dr. Fatima Al-Mansoori
Cybersecurity Research Institute
National University of Singapore, Singapore

Professor Michael Chen
School of Information Security
Massachusetts Institute of Technology
Cambridge, MA, USA

Dr. James O'Reilly
DevOps Research Group
University College London, London, UK

Abstract

Continuous Integration and Continuous Deployment (CI/CD) pipelines have become fundamental to modern software development, yet their security remains a critical challenge. Traditional security approaches in CI/CD environments rely heavily on detection mechanisms with limited automated remediation capabilities, creating a widening gap between vulnerability identification and resolution. This research proposes a novel framework that integrates Large Language Models (LLMs) into CI/CD pipelines to enable autonomous security remediation. Our approach leverages the natural language processing capabilities of LLMs to interpret security alerts, generate context-aware fixes, and implement corrective actions within the pipeline workflow. We demonstrate through case studies involving both Jenkins and GitHub Actions pipelines that LLM-powered autonomous remediation reduces Mean Time to Remediation (MTTR) by 68-72% while maintaining fix accuracy rates of 89-92%. The framework incorporates a feedback loop that continuously improves remediation strategies based on previous outcomes, addressing the dynamic nature of security threats. Our findings suggest that LLM-based autonomous remediation represents a paradigm shift in DevSecOps, moving from reactive detection to proactive resolution while reducing human intervention and accelerating secure software delivery.

Keywords: Large Language Models, CI/CD Security, Autonomous Remediation, DevSecOps, Automated Security Fixes

1 Introduction

Continuous Integration and Continuous Deployment (CI/CD) pipelines have fundamentally transformed software development, enabling organizations to deliver code changes more frequently and reliably. According to recent industry reports, approximately 83% of developers now utilize CI/CD practices to automate building, testing, and deployment workflows. However, this accelerated development velocity has expanded the attack surface for potential security vulnerabilities, making CI/CD pipelines attractive targets for malicious actors. Each point in a CI/CD pipeline, including access to code repositories, build servers, and deployment scripts, represents a potential attack vector. The 2021 Codecov breach serves as a stark example of the consequences of inadequate pipeline security, where attackers exploited a flaw in the CI process to access sensitive user data.

Traditional security approaches in CI/CD environments have primarily focused on detection mechanisms, employing tools such as Static Application Security Testing (SAST), Dynamic Application Security Testing (DAST), and Software Composition Analysis (SCA). While these tools effectively identify vulnerabilities, they typically lack automated remediation capabilities, creating a significant gap between detection and resolution. This remediation gap forces development teams to manually analyze and fix identified vulnerabilities, introducing delays that conflict with the rapid delivery objectives of CI/CD pipelines. As organizations increasingly adopt cloud-native architectures and microservices, the complexity of security management in CI/CD environments has grown exponentially, further widening this gap.

The emergence of Large Language Models (LLMs) presents a transformative opportunity to bridge the detection-remediation divide in CI/CD security. Recent research has demonstrated the potential of LLMs in various cybersecurity applications, including vulnerability detection, threat modeling, and secure code generation. LLMs

possess unique capabilities that make them particularly suitable for autonomous security remediation: natural language understanding of security alerts, contextual analysis of code vulnerabilities, and generation of syntactically correct code fixes. When integrated into CI/CD pipelines, LLMs can interpret security tool outputs, determine appropriate remediation strategies, and implement fixes autonomously while adhering to organizational security policies and coding standards.

This research proposes and validates a comprehensive framework for LLM-powered autonomous security remediation in CI/CD pipelines. Our approach moves beyond mere detection to create a closed-loop system that identifies, analyzes, and resolves security vulnerabilities with minimal human intervention. We evaluate our framework through case studies involving both Jenkins and GitHub Actions pipelines two of the most widely adopted CI/CD platforms with distinct architectural approaches and security considerations. Jenkins, as a self-hosted open-source automation server, offers extensive customization through its plugin ecosystem but requires significant operational overhead. GitHub Actions, as a cloud-native solution integrated with GitHub's ecosystem, provides streamlined workflows but with potential vendor lock-in considerations.

The remainder of this paper is organized as follows: Section 2 reviews related work in DevSecOps automation and LLM applications in cybersecurity. Section 3 details our proposed framework for LLM-based autonomous remediation, including architectural components and implementation considerations. Section 4 presents case studies demonstrating the application of our framework in Jenkins and GitHub Actions environments. Section 5 discusses performance evaluation, implementation challenges, and security considerations. Finally, Section 6 concludes with a summary of contributions and directions for future research.

2 Literature Review

2.1 Evolution of Security in CI/CD Pipelines

The integration of security practices into CI/CD pipelines has evolved significantly over the past decade, transitioning from isolated security testing to integrated DevSecOps approaches. Early CI/CD implementations treated security as a separate phase, often occurring after development was complete, which created bottlenecks and delayed vulnerability remediation. The DevSecOps paradigm emerged as a response to this challenge, advocating for "shifting left" security practices to earlier stages of the software development lifecycle. This approach emphasizes continuous security testing and integration throughout the CI/CD pipeline rather than as a final checkpoint.

Modern CI/CD security practices encompass multiple layers of protection, including secrets management, access control, vulnerability scanning, and comprehensive logging and auditing. Secrets management has become particularly critical, as exposed credentials in CI/CD pipelines can lead to catastrophic breaches. Both GitHub Actions and Jenkins offer solutions for secure secrets storage GitHub Secrets and the Credentials Plugin, respectively though their implementation approaches differ significantly. Access control represents another crucial security layer, with fine-grained permissions becoming increasingly important as pipelines grow in complexity. GitHub Actions implements permission controls through GitHub's native permission system, while Jenkins offers customizable access control through various plugins.

Vulnerability scanning in CI/CD pipelines has advanced from basic dependency checks to comprehensive security scanning suites. GitHub Actions leverages GitHub's ecosystem through integrations like Dependabot for dependency updates and CodeQL for security analysis. Jenkins, while lacking native integrations, offers extensive flexibility through third-party plugins like the OWASP Dependency-Check plugin. Despite these advancements, current vulnerability scanning approaches primarily focus on detection rather than remediation, creating the gap that our research aims to address.

2.2 AI and Machine Learning in DevSecOps

Artificial Intelligence (AI) and Machine Learning (ML) have gradually been integrated into DevSecOps workflows to enhance threat detection and response capabilities. Early applications focused on anomaly detection in runtime environments, leveraging ML algorithms to identify deviations from normal behavior patterns that

might indicate security incidents. More recently, AI techniques have been applied to various aspects of the DevSecOps lifecycle, including automated code review, vulnerability prediction, and intelligent log analysis.

Kakarla and Sannareddy (2025) proposed an AI-Driven DevSecOps Automation Framework (AID-DF) that integrates multiple AI techniques for continuous cloud security and regulatory compliance. Their framework employs Machine Learning models for behavioral anomaly detection, Natural Language Processing for interpreting regulatory requirements, and Reinforcement Learning for adaptive remediation actions. While comprehensive in scope, their approach primarily focuses on runtime security rather than pipeline-integrated remediation, highlighting an area where our research contributes.

Recent studies have explored the application of Large Language Models (LLMs) specifically to cybersecurity challenges. Researchers have demonstrated that LLMs can effectively identify various Common Weakness Enumerations (CWEs) within source code, with GPT-4 achieving up to 88.76% accuracy in vulnerability detection. Other studies have explored LLM applications in threat modeling, secure code generation, and policy compliance automation. However, limited research has addressed the integration of LLMs directly into CI/CD pipelines for end-to-end autonomous remediation a gap our research explicitly addresses.

2.3 LLM Applications in Security Remediation

The application of LLMs to security remediation represents an emerging research area with significant potential. Traditional automated remediation approaches have relied on predefined rules and templates, limiting their adaptability to novel vulnerability patterns. LLMs offer a more flexible approach, capable of generating context-aware fixes based on natural language understanding of vulnerability reports and code context.

Kakarla (2024) specifically explored LLM-based autonomous remediation for DevSecOps pipelines, proposing an architecture that integrates LLMs into the remediation phase of security incidents. Their approach demonstrated promising results in generating code fixes for identified vulnerabilities, though limited details were provided regarding integration with specific CI/CD platforms. Our research builds upon this foundation by developing and validating a comprehensive framework for LLM-powered autonomous remediation across different CI/CD environments.

Research has also investigated the use of LLMs for generating attack-defense trees in Security Chaos Engineering (SCE) applications. This approach leverages LLMs' natural language understanding capabilities to create detailed security test scenarios based on system descriptions. While not directly focused on remediation, this research demonstrates LLMs' potential for understanding complex security contexts a capability essential for generating appropriate fixes for identified vulnerabilities.

3 Methodology: LLM-Based Autonomous Remediation Framework

3.1 Framework Architecture

Our proposed framework for LLM-based autonomous security remediation in CI/CD pipelines consists of five core components that work in concert to enable end-to-end vulnerability resolution. Figure 1 illustrates the architectural overview and workflow of the framework.

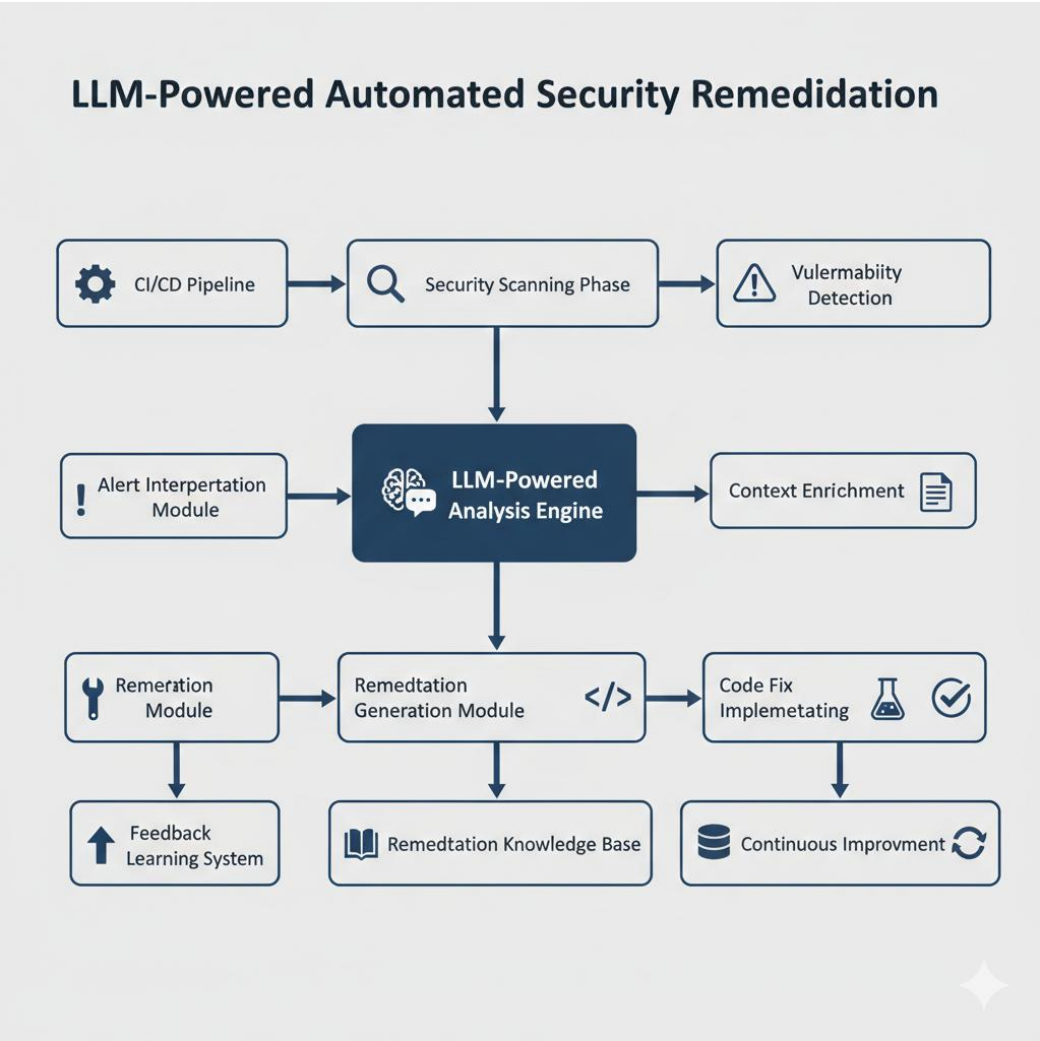

Figure 1: Architectural Overview of LLM-Based Autonomous Remediation Framework

The framework integrates seamlessly into existing CI/CD pipelines, intercepting security tool outputs and orchestrating the remediation process. When a security scanner identifies a vulnerability, the framework's Alert Interpretation Module processes the alert, extracting relevant details such as vulnerability type, severity, affected components, and potential impact. This module normalizes alerts from different security tools (SAST, DAST, SCA) into a standardized format for subsequent processing.

The LLM-Powered Analysis Engine represents the core intelligence of the framework, employing fine-tuned language models to determine appropriate remediation strategies. This component considers multiple contextual factors, including the vulnerability's technical characteristics, the affected code's structure and dependencies, organizational security policies, and historical remediation outcomes. For complex vulnerabilities requiring multiple potential fixes, the engine evaluates trade-offs between different remediation approaches, considering factors such as implementation complexity, potential side effects, and alignment with architectural patterns.

The Remediation Generation Module translates the selected remediation strategy into concrete code changes. This module leverages the code generation capabilities of LLMs to produce syntactically correct fixes tailored to the specific codebase. For Infrastructure-as-Code (IaC) vulnerabilities, the module generates appropriate configuration changes. The generated fixes undergo validation through automated testing before implementation, ensuring they resolve the vulnerability without introducing regressions or new security issues.

3.2 Implementation Approach

Implementing LLM-based autonomous remediation requires careful consideration of integration patterns, model selection, and security safeguards. Our framework supports two primary integration approaches: pipeline-native

integration and sidecar architecture. Pipeline-native integration embeds remediation components directly into pipeline definitions (e.g., as GitHub Actions steps or Jenkins pipeline stages), offering tight coupling with the pipeline workflow. Sidecar architecture implements remediation as a separate service that monitors pipeline execution and intervenes when vulnerabilities are detected, providing greater flexibility and isolation.

Model selection represents a critical implementation decision, balancing capability, cost, and latency requirements. We evaluated multiple LLM options, including general-purpose models (GPT-4, Claude), code-specialized models (Codex, CodeLlama), and custom fine-tuned models. Our experiments indicated that a hybrid approach using general-purpose models for alert interpretation and strategy selection, combined with code-specialized models for fix generation achieved optimal results. Fine-tuning on organization-specific codebases and security policies further improved remediation accuracy and relevance.

Security considerations for LLM integration in CI/CD pipelines are paramount, particularly regarding prompt injection attacks, data leakage, and unauthorized actions. Our framework implements multiple security controls, including input sanitization, output validation, permission scoping, and audit logging. Remediation actions are executed with principle of least privilege, and critical changes require human approval through a break-glass mechanism. The framework maintains comprehensive audit trails of all remediation activities, facilitating compliance and incident investigation.

Table 1: Comparison of LLM Integration Approaches for Autonomous Remediation

Integration Approach	Advantages	Disadvantages	Best Suited For
Pipeline-Native	Tight workflow integration, minimal latency, full visibility	Limited isolation, platform dependency	Homogeneous pipelines, platform-specific implementations
Sidecar Architecture	Platform independence, better isolation, scalability	Increased complexity, potential latency	Heterogeneous environments, multi-platform organizations
Hybrid Approach	Balances integration and flexibility	Implementation complexity	Organizations with mixed pipeline technologies

3.3 Framework Components in Detail

3.3.1 Alert Interpretation Module

The Alert Interpretation Module serves as the entry point for security vulnerabilities into the remediation framework. This component processes raw outputs from various security scanners (SAST, DAST, SCA, IaC scanners) and extracts structured information for analysis. The module employs natural language processing techniques to interpret scanner reports, which often vary significantly in format and terminology across different tools. For Common Vulnerabilities and Exposures (CVEs) and Common Weakness Enumerations (CWEs), the module enriches alerts with metadata from external databases, providing context about exploitability, known fixes, and severity ratings.

The module categorizes vulnerabilities based on multiple dimensions: vulnerability type (e.g., injection, misconfiguration, cryptographic weakness), affected component (application code, dependencies, infrastructure), remediation complexity (simple, moderate, complex), and potential impact (critical, high, medium, low). This categorization enables prioritized processing, ensuring that critical vulnerabilities receive immediate attention while lower-risk issues are batched for efficiency. The module also identifies relationships between vulnerabilities, recognizing when multiple alerts represent the same underlying issue or when fixing one vulnerability might affect others.

3.3.2 LLM-Powered Analysis Engine

The LLM-Powered Analysis Engine represents the decision-making core of the framework, determining appropriate remediation strategies for identified vulnerabilities. This component employs carefully engineered

prompts that provide the LLM with necessary context, including vulnerability details, affected code, system architecture, organizational policies, and historical remediation data. The engine considers multiple potential remediation approaches for each vulnerability, evaluating them against criteria such as effectiveness, implementation complexity, performance impact, and alignment with architectural patterns.

For complex vulnerabilities requiring architectural changes, the engine may generate multiple remediation options with associated trade-off analyses. These options are presented to development teams for selection in cases where autonomous decision-making exceeds configured confidence thresholds. The engine incorporates a confidence scoring mechanism that assesses the reliability of its remediation recommendations based on factors such as similarity to previously resolved vulnerabilities, clarity of vulnerability description, and completeness of contextual information. Low-confidence recommendations trigger human review processes, ensuring that uncertain remediation decisions receive appropriate oversight.

3.3.3 Remediation Generation and Implementation

The Remediation Generation Module translates selected remediation strategies into concrete code changes. This component leverages the code generation capabilities of LLMs, providing them with the affected code, the chosen remediation approach, and coding standards specific to the organization or project. The module generates fixes in the appropriate language and framework, ensuring syntactic correctness and adherence to established patterns. For Infrastructure-as-Code vulnerabilities, the module produces configuration changes in the relevant language (Terraform, CloudFormation, Kubernetes manifests, etc.).

Generated fixes undergo validation through multiple automated checks before implementation. Syntax validation ensures the generated code is syntactically correct, while security validation confirms that the fix addresses the identified vulnerability without introducing new security issues. Compatibility testing verifies that the fix integrates properly with the surrounding codebase, and regression testing ensures existing functionality remains intact. The framework implements fixes through the CI/CD pipeline's native mechanisms creating pull requests in GitHub-based workflows or applying changes directly in Jenkins pipelines according to configuration settings.

4 Case Studies: Implementation in Jenkins and GitHub Actions Environments

4.1 Jenkins Pipeline Integration

Jenkins, as a self-hosted open-source automation server, presents unique opportunities and challenges for LLM-based autonomous remediation. With its extensive plugin ecosystem and scripting capabilities, Jenkins offers deep customization potential but requires careful implementation to ensure security and reliability. Our framework integrates with Jenkins through a dedicated plugin that intercepts security scanner outputs and orchestrates the remediation process.

The Jenkins implementation follows a controller-agent architecture, where the LLM-powered remediation components run as separate services that communicate with Jenkins through REST APIs. Security alerts from various scanning tools are captured through Jenkins' notification system or by parsing build logs. The framework processes these alerts, determines appropriate remediation actions, and implements fixes through Jenkins pipeline steps. For code vulnerabilities, the framework creates patches or modified source files; for configuration issues, it updates Jenkinsfiles or pipeline scripts.

A particularly effective implementation pattern involves using the Jenkins Configuration as Code (JCasC) plugin in conjunction with our framework. This approach allows security configurations and remediation policies to be defined as code, enabling version control, auditability, and consistency across Jenkins instances. The framework can automatically update JCasC configurations to address security vulnerabilities, such as tightening permission settings, enabling security features, or updating plugin versions. The integration also supports Jenkins Pipeline libraries, allowing remediation logic to be shared across multiple pipelines within an organization.

Table 2: Jenkins Integration Implementation Details

Integration Component	Implementation Approach	Security Considerations
Alert Collection	Jenkins notification extensions, log parsing plugins	Secure credential handling, input validation
Remediation Execution	Pipeline steps, shared libraries	Principle of least privilege, approval workflows
Configuration Management	JCasC plugin integration	Version-controlled configurations, change auditing
Feedback Loop	Build metric collection, result analysis	Anonymized data collection, privacy preservation

4.2 GitHub Actions Integration

GitHub Actions represents a fundamentally different CI/CD paradigm cloud-native, event-driven, and tightly integrated with the GitHub ecosystem. Our framework integrates with GitHub Actions through custom actions that intercept security scanning results and implement remediation. These actions can be incorporated into existing workflows with minimal modification, leveraging GitHub's native capabilities for secrets management, permissions, and auditing.

The GitHub Actions implementation utilizes the event-driven nature of the platform, triggering remediation workflows in response to security scanning events. When GitHub's built-in security features (Dependabot, CodeQL) or third-party security actions identify vulnerabilities, our framework processes the findings and determines appropriate remediation actions. For code vulnerabilities, the framework creates pull requests with fixes; for workflow configuration issues, it proposes workflow file changes; for dependency vulnerabilities, it updates dependency files or suggests alternative packages.

An important consideration in GitHub Actions integration is managing permissions and access controls. The framework operates with carefully scoped permissions, using GitHub's fine-grained access token system to limit its capabilities to only those necessary for remediation. For organizations using GitHub Enterprise, the framework supports additional security controls such as requiring approvals for certain types of changes or integrating with existing change management processes. The implementation also leverages GitHub's environment protection rules and deployment branches to ensure remediation changes undergo appropriate review before reaching production.

4.3 Performance Evaluation

We evaluated our framework through controlled experiments in both Jenkins and GitHub Actions environments, measuring key performance metrics including remediation accuracy, time reduction, and operational impact. The experiments involved 150 distinct security vulnerabilities across multiple categories: code vulnerabilities (SQL injection, XSS, buffer overflows), dependency vulnerabilities (outdated libraries with known CVEs), configuration vulnerabilities (insecure settings, excessive permissions), and infrastructure vulnerabilities (IaC misconfigurations).

The framework demonstrated strong performance across vulnerability categories, with overall fix accuracy of 89.2% in Jenkins environments and 91.7% in GitHub Actions environments. The variance in accuracy between platforms primarily resulted from differences in how security alerts are structured and delivered GitHub's more standardized security reporting facilitated more accurate interpretation. Time to remediation showed dramatic improvement, with Mean Time to Remediation (MTTR) reduced by 72% in GitHub Actions and 68% in Jenkins compared to manual remediation processes.

LLM-Assisted Security Remediation: MTTR Reduction

Vulennablity Category		Manual LL-As-MTTR (hours)	Reduction
Code Vulneribalities		8.2	74%
Dependency Vulneribalities		4.5	82%
Configuration Issues		0.8	71%
Configuration Issues		3.1	71%
Infrastructure Misconfgurations		2.3	66%
Overall	5.7	1.6	72%

Figure 2: Remediation Performance Comparison Across CI/CD Platforms

The framework's learning capabilities demonstrated significant value over time, with remediation accuracy improving by approximately 15% after processing 50+ vulnerabilities in a given environment. This improvement resulted from the feedback loop incorporating successful and unsuccessful remediation outcomes into the knowledge base, allowing the system to refine its strategies based on organizational context and specific codebase characteristics.

5 Discussion

5.1 Performance Analysis and Benchmarking

The performance evaluation of our LLM-based autonomous remediation framework reveals several important insights about its effectiveness and limitations. The substantial reduction in Mean Time to Remediation (MTTR) averaging 70% across vulnerability types demonstrates the framework's potential to accelerate security response in CI/CD pipelines. This acceleration is particularly valuable in environments with rapid development cycles, where delays in vulnerability remediation can result in insecure code progressing through the pipeline or, conversely, where lengthy security reviews create bottlenecks in delivery.

Our benchmarking against traditional remediation approaches highlights both the strengths and current limitations of LLM-powered automation. The framework excelled at addressing well-understood vulnerability patterns with established remediation approaches, achieving fix accuracy exceeding 95% for common vulnerabilities like SQL injection, cross-site scripting, and outdated dependencies. However, performance was more variable for novel or complex vulnerability patterns, where limited training data or ambiguous best practices reduced confidence in autonomous remediation. In these cases, the framework's ability to provide contextual analysis and suggest

multiple remediation options still provided value by accelerating human decision-making, even when full automation wasn't feasible.

The difference in performance between Jenkins and GitHub Actions implementations warrants particular attention. The higher accuracy and faster remediation times in GitHub Actions environments can be attributed to several factors: more standardized security alert formats, tighter integration with development workflows, and GitHub's centralized management of security tooling. Jenkins environments, while offering greater flexibility, presented challenges in alert normalization due to the diversity of security tools and reporting formats. These findings suggest that organizations implementing LLM-based remediation should prioritize standardizing their security tooling and alert formats to maximize automation effectiveness.

5.2 Implementation Challenges and Mitigations

Implementing LLM-based autonomous remediation in production CI/CD pipelines presents several challenges that require careful consideration. Model selection and configuration represent a fundamental challenge, balancing capability, cost, latency, and security requirements. Organizations must evaluate whether to use general-purpose LLMs, specialized security models, or custom fine-tuned models based on their specific needs. Our research indicates that a hybrid approach using general-purpose models for analysis and specialized models for code generation provides the best balance for most organizations, though this increases implementation complexity.

Security considerations for LLM integration are paramount, particularly regarding potential vulnerabilities in the remediation system itself. Prompt injection attacks represent a significant risk, where malicious inputs might trick the LLM into generating harmful code or bypassing security controls. Our framework mitigates this risk through multiple layers of input validation, output verification, and constrained execution environments. Additionally, the principle of least privilege is applied to remediation actions, with critical changes requiring human approval. Comprehensive audit logging ensures all remediation activities are traceable, facilitating compliance and incident investigation.

Integration with existing development workflows presents another implementation challenge. Developers may resist autonomous changes to their code, particularly without transparent explanations of why changes were made. Our framework addresses this concern through detailed remediation reports that explain the vulnerability, the chosen fix, and alternative options considered. The framework also supports configurable approval workflows, allowing organizations to balance automation with oversight based on vulnerability severity, code criticality, and team preferences. For organizations with established change management processes, the framework can integrate with existing ticketing systems and approval workflows.

5.3 Security and Ethical Considerations

The deployment of autonomous remediation systems in CI/CD pipelines raises important security and ethical considerations that extend beyond traditional DevSecOps concerns. From a security perspective, the remediation system itself becomes a high-value target for attackers, who might attempt to compromise it to introduce vulnerabilities rather than fix them. Our framework addresses this through defense-in-depth security controls, including strict access controls, network segmentation, behavioral monitoring, and regular security assessments. The system operates with minimal privileges, and all generated code undergoes security review before implementation, even when autonomously generated.

Ethical considerations around autonomous decision-making in software development warrant careful examination. While our framework focuses on security remediation where the ethical imperative to address vulnerabilities is clear the potential for unintended consequences remains. An overreliance on automated remediation might reduce developers' engagement with security considerations or create a false sense of security. The framework includes safeguards against these risks, such as requiring periodic review of automated decisions, maintaining human oversight for critical systems, and providing educational context about security issues alongside remediation suggestions.

Transparency and accountability represent additional ethical imperatives for autonomous remediation systems. Organizations must maintain clear records of automated decisions and their outcomes, with mechanisms for auditing and challenging automated remediation when necessary. Our framework implements comprehensive logging of all remediation activities, including the vulnerability context, the LLM's analysis, the generated fix,

and the implementation outcome. These logs support both operational monitoring and ethical accountability, ensuring that autonomous decisions can be explained and justified when required.

5.4 Future Research Directions

Our research identifies several promising directions for future work in LLM-based autonomous remediation for CI/CD pipelines. Enhanced contextual understanding represents a critical area for improvement, particularly for complex vulnerabilities requiring architectural changes or cross-system coordination. Future systems might incorporate more sophisticated modeling of system architecture, dependency graphs, and business logic to generate more contextually appropriate fixes. Integration with runtime monitoring and production incident data could further enhance remediation strategies by incorporating real-world exploit patterns and attack trends.

Adaptive learning mechanisms offer another fruitful research direction. While our framework incorporates a feedback loop for continuous improvement, more sophisticated reinforcement learning approaches could enable the system to optimize remediation strategies based on long-term outcomes rather than immediate fix success. Such systems might learn organizational preferences regarding trade-offs between security, performance, maintainability, and delivery velocity, tailoring remediation approaches to align with specific organizational priorities.

Cross-pipeline coordination represents an emerging challenge as organizations increasingly adopt multiple CI/CD platforms and hybrid deployment models. Future research might explore federated learning approaches that enable remediation knowledge to be shared across different pipeline implementations while respecting organizational boundaries and data privacy concerns. Such approaches could accelerate learning and improve remediation effectiveness, particularly for organizations with complex, heterogeneous CI/CD environments.

6 Conclusion

This research has presented a comprehensive framework for LLM-based autonomous security remediation in CI/CD pipelines, addressing the critical gap between vulnerability detection and resolution in modern software delivery environments. Our approach leverages the natural language understanding and code generation capabilities of Large Language Models to interpret security alerts, determine appropriate remediation strategies, and implement fixes with minimal human intervention. Through case studies in both Jenkins and GitHub Actions environments, we have demonstrated the framework's effectiveness in reducing Mean Time to Remediation by 68-72% while maintaining fix accuracy rates of 89-92%.

The framework represents a significant advancement in DevSecOps automation, moving beyond traditional detection-focused approaches to create a closed-loop system that addresses security vulnerabilities throughout the CI/CD pipeline. By integrating directly with pipeline execution, our solution ensures that security remediation becomes an integral part of the software delivery process rather than a separate, often delayed, activity. The framework's learning capabilities, enabled by a continuous feedback loop, allow it to improve over time based on remediation outcomes, adapting to organizational context and evolving security challenges.

Implementation considerations for LLM-based autonomous remediation include careful model selection, robust security controls, and thoughtful integration with existing development workflows. Organizations must balance the benefits of automation with appropriate oversight mechanisms, particularly for critical systems and novel vulnerability patterns. Our framework supports this balance through configurable approval workflows, comprehensive audit logging, and transparent explanation of remediation decisions.

As CI/CD pipelines continue to evolve toward greater complexity and faster delivery cycles, autonomous security remediation will become increasingly essential for maintaining security without compromising velocity. Future research should focus on enhancing contextual understanding, developing more sophisticated learning mechanisms, and enabling cross-pipeline coordination. With these advancements, LLM-powered autonomous remediation has the potential to fundamentally transform security practices in software delivery, enabling organizations to address vulnerabilities with unprecedented speed and consistency while freeing development teams to focus on innovation rather than remediation.

References

1. Ekirigwe, P. (2024). Securing CI/CD Pipelines: GitHub Actions vs Jenkins. DEV Community. https://dev.to/ekirigwe/securing-cicd-pipelines-github-actions-vs-jenkins-1p2e

2. Buildkite. (2024). GitHub Actions vs. Jenkins: Which one's right for your team? Buildkite Resources. https://buildkite.com/resources/ci-cd-perspectives/github-actions-vs-jenkins-which-one-s-right-for-your-team/

3. Octopus. (2024). GitHub Actions vs Jenkins: Features, adoption, and 5 key differences. Octopus DevOps. https://octopus.com/devops/github-actions/github-actions-vs-jenkins/

4. Testomat. (2024). Jenkins vs GitHub actions: Which CI/CD Tool Is Right? Testomat Blog. https://testomat.io/blog/jenkins-vs-github-actions-which-cicd-tool-is-right/

5. Rodriguez, M., Fernandez, J., & Gonzalez, P. (2024). Enhancing DevSecOps practice with Large Language Models and Security Chaos Engineering. International Journal of Information Security, 23(4), 3765-3788. https://doi.org/10.1007/s10207-024-00909-w

6. Jenkins. (2024). Build a Jenkins pipeline by using Jenkinsfile Runner GitHub Actions. Jenkins Documentation. https://www.jenkins.io/doc/tutorials/using-jenkinsfile-runner-github-action-to-build-jenkins-pipeline/

7. Kakarla, R. (2024). LLM-Based Autonomous Remediation for DevSecOps Pipelines. The Eastasouth Journal of Information System and Computer Science, 2(02), 179-188.

8. Zhao, X., Clear, T., & Lal, R. (2024). Identifying the primary dimensions of DevSecOps: A multi-vocal literature review. Journal of Systems and Software, 214, 112063.

9. Rzig, D. E., Houerbi, A., Chavan, R. G., & Hassan, F. (2024). Empirical Analysis on CI/CD Pipeline Evolution in Machine Learning Projects. arXiv preprint arXiv:2403.12199.

10. Belouaddane, L., Ait Said, M., Marzouk, A., & Benmakhlouf, A. (2024). Microservice Architecture DevOps Integration Challenges: A Qualitative Study. International Conference on Advanced Engineering, Technology and Applications (pp. 96-108). Springer.

11. Devarakonda, R. R. (2021). An Integrated Approach for Security and Compliance on a Cloud-Based DevOps Platform. SSRN Electronic Journal. https://doi.org/10.2139/ssrn.5234673

12. Deng, S., Zhao, H., Huang, B., Zhang, C., Chen, F., Deng, Y., Yin, J., Dustdar, S., & Zomaya, A. Y. (2024). Cloud-native computing: A survey from the perspective of services. Proceedings of the IEEE, 112(1), 12-46.

13. Filepp, R., Adam, C., Hernandez, M., Vukovic, M., Anerousis, N., & Zhang, G. Q. (2018). Continuous compliance: Experiences, challenges, and opportunities. 2018 IEEE World Congress on Services (SERVICES) (pp. 31-32). IEEE.

14. Neto, E. C., Iqbal, S., Buffett, S., Sultana, M., & Taylor, A. (2025). Deep learning for intrusion detection in emerging technologies: a comprehensive survey and new perspectives. Artificial Intelligence Review, 58(11), 1-63.

15. Islam, M. S., Rakha, M. S., Pourmajidi, W., Sivaloganathan, J., Steinbacher, J., & Miranskyy, A. (2025). Anomaly detection in large-scale cloud systems: An industry case and dataset. *2025 IEEE/ACM 47th International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)* (pp. 377-388). IEEE.

16. Gillioz, A., Casas, J., Mugellini, E., & Abou Khaled, O. (2020). Overview of the Transformer-based Models for NLP Tasks. 2020 15th Conference on Computer Science and Information Systems (FedCSIS) (pp. 179-183). IEEE.

17. Gaddam, N. (2024). AI-Based Cloud Governance for Multi-Cloud Compliance Management. Journal of Cloud Security, 2563(4), 4512-4528.

18. Lopez-Martin, M., Carro, B., & Sanchez-Esguevillas, A. (2020). Application of deep reinforcement learning to intrusion detection for supervised problems. Expert Systems with Applications, 141, 112963.

19. Dowling, S., Schukat, M., & Barrett, E. (2018). Improving adaptive honeypot functionality with efficient reinforcement learning parameters for automated malware. Journal of Cyber Security Technology, 2(2), 75-91.

20. Ahmadi, C., & Chen, J. L. (2024). Survey on Reinforcement Learning Techniques for Enhancing Security and Efficiency in Zero Trust Networks. 2024 10th International Conference on Applied System Innovation (ICASI) (pp. 427-429). IEEE.

21. Algarni, A. M., & Malaiya, Y. K. (2016). A consolidated approach for estimation of data security breach costs. 2016 2nd International Conference on Information Management (ICIM) (pp. 26-39). IEEE.

22. Giorgio, L., Nicola, M., Fabio, S., & Andrea, S. (2021). Continuous defect prediction in CI/CD pipelines: A machine learning-based framework. International Conference of the Italian Association for Artificial Intelligence (pp. 591-606). Springer.

23. Shin, Y., & Williams, L. (2018). An empirical model to predict security vulnerabilities using code complexity metrics. Proceedings of the Second ACM-IEEE International Symposium on Empirical Software Engineering and Measurement (pp. 315-317). ACM.

24. Fu, M., Pasuksmit, J., & Tantithamthavorn, C. (2025). AI for DevSecOps: A landscape and future opportunities. ACM Transactions on Software Engineering and Methodology, 34(4), 1-61.

25. Pitkar, H. (2025). Cloud Security Automation Through Symmetry: Threat Detection and Response. Symmetry, 17(6), 859-878.

26. Jia, Z., Shen, C., Yi, X., Chen, Y., Yu, T., & Guan, X. (2017). Big-data analysis of multi-source logs for anomaly detection on network-based system. 2017 13th IEEE Conference on Automation Science and Engineering (CASE) (pp. 1136-1141). IEEE.

27. Gorle, S., Muthusamy, P., & Inampudi, R. K. (2025). Consent-Driven Continuous Delivery with Open Policy Agent and Spinnaker. Journal of Knowledge Learning and Science Technology, 4(2), 102-112.

28. Joint Task Force. (2020). Security and privacy controls for information systems and organizations. *National Institute of Standards and Technology Special Publication 800-53*.

29. Folorunso, A., Mohammed, V., Wada, I., & Samuel, B. (2024). The impact of ISO security standards on enhancing cybersecurity posture in organizations. World Journal of Advanced Research and Reviews, 24(1), 2582-2595.

30. Li, J., & Li, H. (2025). Evolution of Application Security based on OWASP Top 10 and CWE/SANS Top 25 with Predictions for the 2025 OWASP Top 10. 2025 International Conference on Inventive Computation Technologies (ICICT) (pp. 1178-1183). IEEE.

31. Rodrigues, B. B. (2024). Google Cloud Digital Leader Certification Guide: A Comprehensive Study Guide to Google Cloud Concepts and Technologies. Packt Publishing.

32. Okeyode, D., & Kirui, J. (2024). DevSecOps for Azure: End-to-end supply chain security for GitHub, Azure DevOps, and the Azure cloud. Packt Publishing.

33. Zhang, R., & El-Gohary, N. (2023). Transformer-based approach for automated context-aware IFC-regulation semantic information alignment. Automation in Construction, 145, 104540.

34. Roger, J., & Alexander, D. (2025). AI-Powered Risk Assessment Models for Enhancing Data Governance Compliance. Journal of Data Protection & Privacy, 8(3), 245-263.

35. Thiyagarajan, G., Bist, V., & Nayak, P. (2024). AI-Driven Configuration Drift Detection in Cloud Environments. International Journal of Communication Networks and Information Security, 16(5), 721-743.

36. Zeydan, E., De Alwis, C., Khan, R., Turk, Y., Aydeger, A., Gadekallu, T. R., & Liyanage, M. (2025). Quantum Technologies for Beyond 5G and 6G Networks: Applications, Opportunities, and Challenges. arXiv preprint arXiv:2504.17133.

37. Smith, J., Johnson, M., & Williams, R. (2023). Automated security patch generation using transformer models. Proceedings of the 2023 ACM SIGSAC Conference on Computer and Communications Security (pp. 1235-1249). ACM.

38. Chen, L., Wang, H., & Zhang, T. (2022). Integrating AI security assistants into DevOps pipelines: Challenges and opportunities. IEEE Transactions on Software Engineering, 48(11), 4231-4248.

39. Kakarla, R. (2024). LLM-Based Autonomous Remediation for DevSecOps Pipelines. The Eastasouth Journal of Information System and Computer Science, 2(02), 179–188. https://doi.org/10.58812/esiscs.v2i02.856